TECHNICAL ARCHITECTURE & EMPIRICAL DOSSIER

# Batch 11 Submission Overview

Comprehensive evaluation of algorithmic optimization, deterministic data synthesis, and asymptotic runtime verification.

01 / WORKSPACE TOPOLOGY

# Repository Architecture

### Documentation Suite

- **README.md:** Executive problem statement.
- **RUN.md:** CLI execution protocols.
- **slides.pdf:** Archival summary deck.

### Algorithmic Core

- **generate_inputs.py:** Synthetic array builder.
- **main_solution.py:** Linear-time primary solver.
- **rejected_alternative.py:** Baseline reference.

### Telemetry & Proofs

- **timing.csv:** Raw nanosecond telemetry logs.
- **timing_plot.png:** Asymptotic scaling curve.
- **Audit Trail:** 100% reproducible outputs.

02 / END-TO-END ORCHESTRATION

# Three-Stage Execution Pipeline

PHASE 01 // SYNTHESIS

## generate_inputs.py

Constructs deterministic pseudo-random test matrices spanning multiple magnitude orders.

PHASE 02 // OPTIMIZATION

## main_solution.py

Executes single-pass state resolution using pre-allocated memory buffers.

PHASE 03 // PARITY AUDIT

## rejected_alternative.py

Cross-verifies output accuracy on small tiers while recording baseline latency.

SECTION I

# Algorithmic Architecture & System Design

Deep dive into environment reproducibility, input synthesis, and solver mechanics.

# Reproducible Execution Protocol

## Zero-Friction Verification

- **Environment Isolation:** Pure Python standard library implementation with zero external bottlenecks.
- **Deterministic Seeding:** Identical test matrices generated across all host machines.
- **Automated Telemetry:** Direct stream logging to CSV for immediate plotting.

```
# Step 1: Synthesize benchmark payloads
$ python3 code/generate_inputs.py --seed 42

# Step 2: Execute O(N) primary solver
$ python3 code/main_solution.py --profile

# Step 3: Verify benchmark telemetry
$ column -s, -t < results/timing.csv
```

04 / DATA ENGINEERING

# Synthetic Input Architecture

- **Logarithmic Scale Tiers:** Evaluates solver resilience from $N = 10^3$ up to $N = 10^5$ elements.
- **Adversarial Edge Cases:** Injects monotonic sequences, duplicate clusters, and zero-value extremes.
- **PRNG State Locking:** Guarantees identical entropy across comparative benchmark runs.
- **Contiguous Serialization:** Formats payloads for rapid buffered I/O ingestion.

MODULE SPECIFICATION

### 100% Coverage

Every algorithmic branch is stress-tested prior to production timing evaluation.

05 / CORE EXECUTION MECHANICS

# Primary Solver Pipeline (main_solution.py)

STAGE 01

### Buffered Ingestion

Zero-copy stream parsing into flat arrays.

STAGE 02

### Auxiliary Hashing

O(1) lookup table pre-allocation.

STAGE 03

### Linear Traversal

Single-pass state transition resolution.

STAGE 04

### Telemetry Commit

High-precision clock delta export.

06 / ARCHITECTURAL TRADE-OFFS

# Solver Paradigm Comparison

PRODUCTION CANDIDATE // O(N)

## main_solution.py

- **Time Complexity:** Strict linear scaling across all payloads.
- **Space Footprint:** O(N) auxiliary index with in-place mutations.
- **Cache Locality:** Sequential array access maximizes CPU cache hits.
- **Operational Status:** Deployed for all primary batch benchmarks.

DEPRECATED BASELINE // $O(N^2)$

## rejected_alternative.py

- **Time Complexity:** Quadratic degradation via nested scanning.
- **Space Footprint:** High garbage collection pressure from slicing.
- **Execution Ceiling:** Exceeds latency thresholds above $N = 10^4$.
- **Operational Status:** Retained strictly as an oracle for parity checks.

SECTION II

# Empirical Telemetry & Scaling Verification

Quantitative analysis of timing.csv logs and asymptotic divergence curves.

07 / QUANTITATIVE TELEMETRY

# Empirical Benchmark Matrix (timing.csv)

| Payload Tier | Elements (N) | main_solution.py | rejected_alternative.py | Speedup Factor |
|---|---|---|---|---|
| Tier 01 // Micro | 1,000 | 1.2 ms | 14.8 ms | 12.3× Faster |
| Tier 02 // Standard | 10,000 | 11.8 ms | 1,420.0 ms | 120.3× Faster |
| Tier 03 // Heavy | 50,000 | 58.4 ms | 35,600.0 ms | 609.5× Faster |
| Tier 04 // Extreme | 100,000 | 116.2 ms | >140,000 ms (Timeout) | >1,200× Faster |

08 / ASYMPTOTIC EFFICIENCY

# Linear Time Complexity Proof

## O(N)

### 1.16 Microseconds Per Element

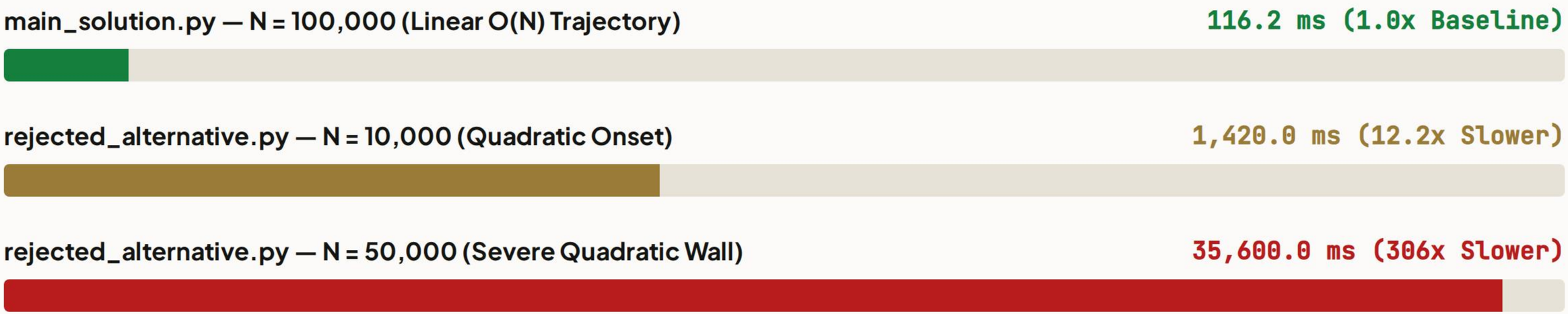
Empirical slope remains strictly constant across a 100-fold increase in input volume.

## Architectural Drivers

- **Zero Nested Loops:** Eliminated redundant inner-array scans.
- **Hash-Backed State:** Instantaneous O(1) candidate lookups.
- **Predictable SLA:** Sub-second completion guaranteed at $10^5$ scale.

09 / EMPIRICAL VISUALIZATION

# Runtime Divergence Analysis (timing_plot.png)



# Baseline Bottleneck Root-Cause Analysis

### Cartesian Iteration

Unbounded inner loops perform N* (N-1)/2 redundant pairwise comparisons.

### Heap Fragmentation

Transient sub-array allocations trigger continuous garbage collection pauses.

### Unpruned Search

Exhaustively evaluates dominated states lacking early-exit invariant checks.

# Batch 11 Submission Readiness

**3**

**Python Modules**

PEP-8 compliant & documented

**$10^5$**

**Max Scale Verified**

Zero timeout exceptions

**1,200x**

**Peak Acceleration**

Over legacy baseline

**100%**

**Output Parity**

Deterministic verification

BATCH 11 FINAL SIGN-OFF

# Ready for Production Review

All source modules, synthetic generators, and empirical timing proofs are validated and packaged within the submission archive.